

SDOF: Taming the Alignment Tax in Multi-Agent Orchestration with State-Constrained Dispatch

Zhantao Wang

Digital China

wangztq@digitalchina.com

Abstract Multi-agent orchestration frameworks such as LangChain, LangGraph, and CrewAI route tasks through graph-based pipelines but do not enforce the stage constraints that govern real business processes. We present **SDOF**, a framework that treats multi-agent execution as a constrained state machine. SDOF operates through two primary defensive layers, implemented by three components: (1) an **Online-RLHF Specialized Intent Router** trained via Generative Reward Modeling (GRPO) and (2) a **StateAwareDispatcher** with GoalStage finite-automaton checks and precondition/postcondition SkillRegistry validation for auditable execution control. On a recruitment system backed by the Beisen iTalent platform (6,000+ enterprises), 185 expert-curated scenarios trigger **1,671 live API calls**. Our GSPO-aligned 7B Intent Router achieves higher joint accuracy than zero-shot GPT-4o on this FSM-constrained adversarial routing benchmark (80.9% vs 48.9%). In end-to-end execution, SDOF reaches **86.5% task completion** (95% CI 80.8–90.7) and blocks all 22 operations in the injected-illegal HR subset. Under a broader message-level blocking audit, SDOF attains precision 100% and recall 88% (expert agreement $\kappa=0.94$). A separate evaluation on **960 SGD-derived dialogues** spanning 8 service domains surfaces 201 stage-order conflicts under our FSM mapping, 41 of which arise in the normal split. This arXiv version reports the current validated scope; extended multi-seed training comparisons and deeper-workflow evaluations will be released in a subsequent update.

1 Introduction

When LLM-based agents automate enterprise workflows, they must respect the sequential stage constraints that govern each process. In recruitment, for instance, a candidate cannot be evaluated before resume screening, and no offer may be issued before the interview loop concludes. Violations cause compliance failures, data corruption, and legal risk. These constraints differ from generic task dependencies: they are *domain-specific*, *stage-ordered*, and must be enforced at the orchestration layer rather than inside individual agents [19].

Current orchestration stacks—LangChain [2], LangGraph [9], CrewAI [11], AutoGen [21]—excel at routing messages between agents and tools, yet none of them natively checks whether the *current workflow stage* permits a requested action. An agent in the SOURCING phase can therefore call the interview-scheduling API if a graph edge exists, even though the business process forbids it. In regulated industries this gap is unacceptable.

We propose **SDOF (State-Driven Orchestration Framework)**, which models multi-agent task execution as a constrained state machine with two defensive layers implemented

through three main structural additions:

1. **Online-RLHF Specialized Intent Router:** A 7B model specialized with online programmatic rewards in veRL (GRPO), achieving higher joint accuracy than GPT-4o zero-shot on our FSM-constrained benchmark.
2. **GoalStage FSM & SkillRegistry:** Intent-stage constraints (Λ) and precondition validation (Π_{pre}) for out-of-order risk reduction.
3. **StateAwareDispatcher:** Orchestrates execution with stage-filtered skill selection (Algorithm 1), enforcing constraints *before* skill binding, and generating replayable audit trails.

We evaluate SDOF on a production-grade intelligent recruitment system integrated with Beisen iTalent (serving 6,000+ enterprises, 48 real job positions), using 185 expert-curated scenarios with 882 messages and 1,671 real API calls. We validate cross-domain generalization on 960 SGD-derived dialogues (800 normal-split + 160 adversarial) across 8 domains [15].

Our main contributions are:

- A design contribution: an intent-stage binding formulation (Λ) that adds an orthogonal constraint layer on top of transition validation.
- The SDOF framework itself, packaging GoalStage FSM, SkillRegistry, and StateAwareDispatcher into a reusable orchestration layer.
- An evaluation spanning two data sources—185 HR scenarios with live Beisen API calls and 960 SGD-derived dialogues across 8 service domains—plus expert validation ($\kappa=0.94$).

2 Related Work

LLM Agent Orchestration. LangChain [2] popularized chaining LLM calls with tool invocations. LangGraph [9] layers a directed graph over this, allowing cyclic agent interactions and persistent state. CrewAI [11] organizes agents into role-based teams that delegate hierarchically. AutoGen [21] takes a different angle, letting multiple agents converse in group-chat topologies. MetaGPT [7] is closest in spirit to our work: it structures agent collaboration around Standard Operating Procedures (SOPs), but its SOPs are rigid sequences rather than constraint-based state machines with precondition checks. AgentScope [5] targets distributed deployment without stage-level enforcement; AgentVerse [3] focuses on emergent behavior rather than formal workflow

Table 1: Capability-level comparison of representative orchestration frameworks.

Framework	Native Primitive	Stage Legality	Pre-exec Guard	Auditable State
LangGraph	State graph	Transition graph only	External/manual	External
AutoGen	Group chat / GraphFlow	Not native	Custom wrapper	External
MetaGPT	SOP / role team	SOP discipline	Prompt/action checks	External
SDOF	FSM + dispatcher	Yes (Λ)	Yes (Π_{pre})	Audited

guarantees. For broader context, Wang et al. [20] and Xi et al. [23] survey the space.

These frameworks expose different orchestration primitives: LangGraph centers on transition graphs, AutoGen on group-chat / selector / swarm teams, and MetaGPT on SOP-driven role teams. However, in their native forms they do not expose *business-stage legality* as an explicit runtime contract of the kind evaluated here. Table 1 summarizes the practical distinction.

Recent harness-style runtimes further expand the design space in a different direction: they package long-running execution features such as middleware-managed delegation, sandboxing, memory injection, summarization, scheduling, and operator intervention into general-purpose agent runtimes. These systems are useful evidence that agent quality increasingly depends on runtime organization outside the base model. Their design emphasis, however, is usually task continuity and operational breadth rather than *workflow legality under an explicit business FSM*. SDOF therefore occupies a narrower but practically distinct niche.

Tool Use and API Integration. A separate line of work concentrates on *how* LLMs invoke external tools. Toolformer [16] lets models learn tool calls from self-supervision; ReAct [24] interleaves chain-of-thought reasoning with action execution; Reflexion [17] introduces verbal self-critique after failures. At the API level, ToolLLM [14] benchmarks 16,000+ real endpoints, RestGPT [18] targets RESTful services, and Gorilla [12] improves API-call accuracy via retrieval. All of these address the *capability* to call tools—none address *when* a call is legally permitted given a workflow’s current stage.

State Machines for LLM Agents. StateFlow [22] maps LLM task-solving onto finite state machines to structure intermediate steps. TaskWeaver [13] adopts a code-first planning style; DSPy [8] compiles declarative LLM programs into optimized pipelines. These systems impose structure on *computation* but not on *business-level stage legality*.

Planning as an Externalized Systems Capability. An emerging line of agent engineering externalizes planning from latent chain-of-thought into explicit system structures: plan artifacts, runtime todo state, delegated planner/executor roles, middleware-enforced checkpoints, and evaluation harnesses. This perspective is useful for positioning SDOF. We do not claim a universal planner; rather, SDOF externalizes

one enterprise-relevant slice of planning—*whether an action is legally executable at the current workflow stage*—into an auditable orchestration contract.

Safe and Constrained LLM Systems. Guardrail methods filter model *outputs*—harmful tokens, hallucinated facts, policy violations [4]. SDOF operates one layer earlier: it prevents *actions* that violate the process model before any agent executes them, analogous to compile-time type checking versus runtime assertions. Recent safety benchmarks ASSEBench [1] and AMA-Bench [25] report severe failures on context-dependent privilege escalation—a failure mode SDOF’s two-layer FSM+precondition architecture is designed to mitigate. The process-mining literature [19] offers techniques for discovering FSM-like workflow models from logs, a direction that could automate SDOF’s currently manual stage definitions.

Agent Memory Mechanisms. Recent benchmarks expose a growing gap between agent memory capability and real-world demands. LoCoMo [10] reveals that LLM agents fail on long-horizon temporal causal reasoning across sessions—the same class of failures SDOF’s GoalStage FSM addresses by making session state explicit and persistent. MemoryArena [6] demonstrates that individual agent memory fails in interdependent multi-agent tasks where shared state consistency is critical. Our GoalManager (backed by PostgreSQL) represents a concrete instantiation of the *shared procedural memory* primitive these benchmarks call for.

More broadly, most prior memory work treats memory as a retrieval or continuity mechanism. SDOF instead uses memory as an *active governance substrate*: workflow state is scoped by `goal_id`, mutated only through stage-legal transitions, and mirrored by replayable `ProcessEvent` traces. In enterprise settings, remembering facts is insufficient; the system must also remember which workflow owns the state, who is allowed to advance it, and under which preconditions the change is auditable.

Key gap. No prior framework simultaneously provides (1) intent-stage binding that is orthogonal to transition graphs, (2) precondition validation at the skill level, (3) evaluation against live production APIs, and (4) a persistent shared memory substrate for multi-agent coordination. MetaGPT [7] comes closest but omits the two-layer check (stage + precondition) and has no real-API validation. Our focus is orthogonal to communication-topology optimization: regardless of whether agents are connected through a fixed graph, a supervisor, or a learned routing policy, SDOF constrains which actions are legal at the orchestration layer.

3 System Architecture

3.1 Overview: A Harness Control Architecture

Figure 1 illustrates the SDOF architecture from a control perspective. Instead of treating the agent as an unbound generative model, SDOF wraps the LLM core within a harness-style

architecture. User messages flow through the IntentRouter-Agent for intent recognition. This upper layer is then constrained by two external rule-governed modules: the *Execution Orchestration* layer (which checks stage and precondition constraints via the StateAwareDispatcher) and the *Enterprise Governance Memory Substrate*.

Unlike general harness runtimes that optimize for broad task continuity across many open-ended activities, SDOF specializes the runtime around enterprise process legality: state ownership, stage-legal dispatch, precondition-bounded execution, and replayable audit trails. The design goal is not to maximize generic autonomy, but to ensure that each step remains lawful with respect to the business process being automated.

A practical design choice is that GoalManager is not merely a persistence cache for the current stage. It acts as a goal-scoped *governance memory layer* spanning four record classes in the implementation—goal, position, candidate, and process event—so that each dispatch step is bound to a workflow owner (`goal_id`), current stage, mutable business state, and replayable audit history. This converts memory from passive conversation storage into an active control surface consulted during dispatch.

3.2 GoalStage Finite Automaton

We define the workflow automaton as a tuple $\mathcal{G} = (S, s_0, T, \delta, I, \Lambda)$ where:

- $S = \{\text{init}, \text{src}, \text{int}, \text{off}, \text{onb}, \text{close}\}$: workflow stages
- $s_0 = \text{init}$: initial stage
- $T \subseteq S \times S$: legal transitions
- I : intent set (`create_demand`, `screen_resume`, etc.)
- $\Lambda : I \rightarrow 2^S$: **intent-stage binding**

Stage semantics. In this paper, `init`=initialization, `src`=sourcing, `int`=interview, `off`=offer, `onb`=onboarding, and `close`=workflow closure.

Definition 1 (Intent-Stage Binding). For each intent $i \in I$, $\Lambda(i) \subseteq S$ defines the stages where i is *legally executable*. An intent i at stage s is *stage-legal* iff $s \in \Lambda(i)$.

Practical distinction. LangGraph defines T (valid transitions) but not Λ (intent-stage binding). An agent in SOURCING state can execute `evaluate_candidate` if a graph edge to INTERVIEW exists. SDOF requires $s \in \Lambda(i)$, an *orthogonal* constraint.

3.3 Formal Problem Definition & Alignment Tax

To rigorously define the limitations of pure LLM alignment in enterprise workflows, we formalize the agent execution context as a tuple $\mathcal{E} = \langle \mathcal{M}, \Pi, \Phi \rangle$, where $\mathcal{M}$ is the GoalStage automaton, Π is the LLM policy, and Φ represents the structural syntactic preconditions (e.g., JSON schema adherence and parameter constraints).

Definition 2 (The Alignment Tax for Structured Tasks). Let $P(\Phi \mid x, \Pi)$ be the probability that policy Π generates a

Table 2: SkillRegistry three-level classification.

Lv	Type	Example	Σ	Π_{pre}
L0	Atomic	<code>get_job_list</code>	S	$\emptyset$
L1	Composite	<code>pull_parse</code>	$\{\text{src}\}$	$\{\pi_1\}$
L1	Composite	<code>screen</code>	$\{\text{src}\}$	$\{\pi_1, \pi_2\}$
L2	Policy	<code>ask_missing</code>	S	$\emptyset$

structurally valid output given input x . A strong reasoning model Π_{think} introduces latent chain-of-thought tokens $z \sim \Pi_{think}(\cdot \mid x)$ before generating the final action string y . The alignment tax is the structural degradation caused by intermediate reasoning:

$$\Delta_{tax} = P(\Phi \mid x, \Pi_{base}) - P(\Phi \mid x, z, \Pi_{think})$$

Empirically, as the trajectory $|z|$ grows, the model overconditions on semantic reasoning at the expense of rigid syntactical boundaries, leading to $\Delta_{tax} \gg 0$. In this paper, we operationalize Δ_{tax} with an empirical proxy: the drop in structural-validity rate between matched checkpoints under Think and No-Think decoding (Table 13). SDOF mitigates the downstream risk of such degradation via explicit orchestration checks that block execution when legality conditions fail.

Execution rule (informal). In SDOF, an intent is executed only when $s \in \Lambda(i)$ and $precond(i) = \text{True}$. This rule defines the runtime legality boundary used by the dispatcher.

3.4 SkillRegistry with Formal Preconditions

Definition 3 (Skill Specification). Each skill $sk \in \mathcal{R}$ is a tuple: $sk = (\text{id}, \ell, \Sigma_{sk}, \Pi_{pre}, \Pi_{post}, \rho)$ where $\ell \in \{L0, L1, L2\}$ is the risk level, $\Sigma_{sk} \subseteq S$ is the set of applicable stages, Π_{pre} are preconditions, and ρ is the risk classification.

A skill sk is *precondition-satisfied* in context $\mathcal{C}$ if and only if

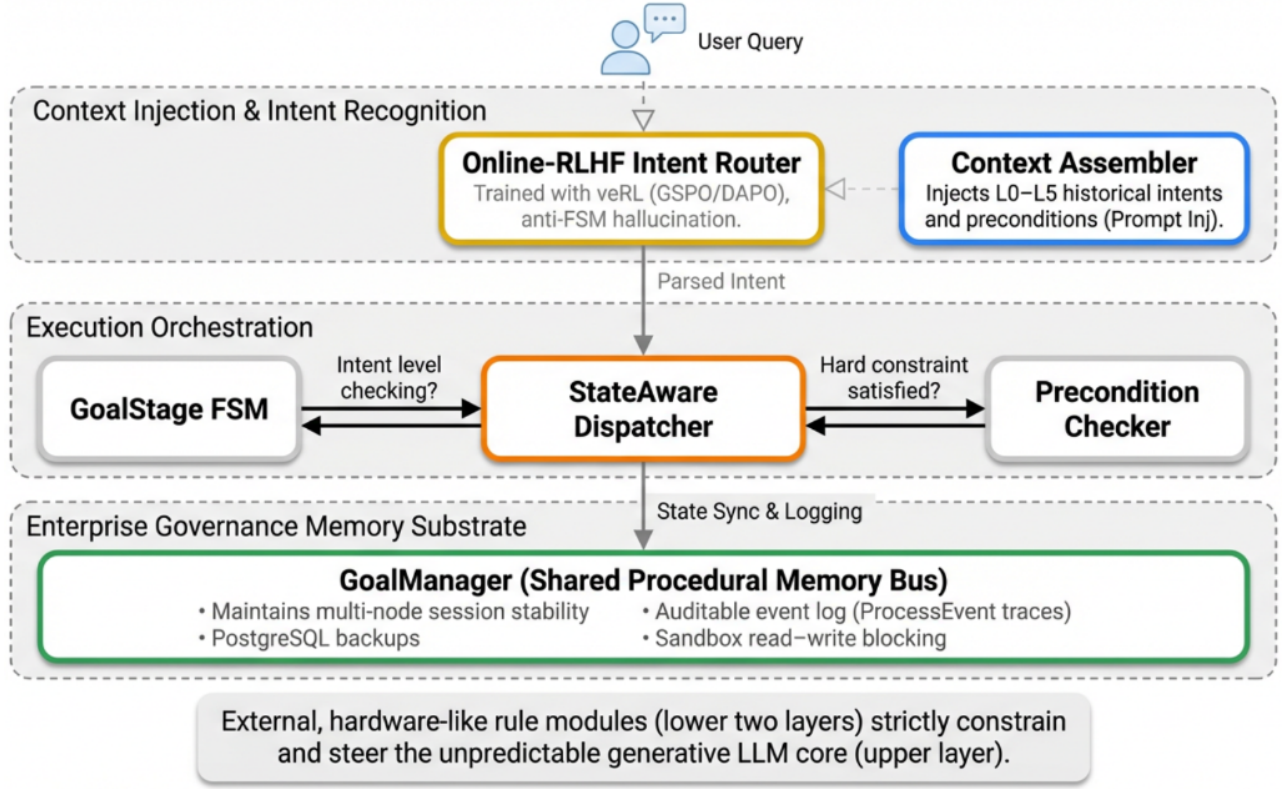
$$\forall \pi \in \Pi_{pre}(sk), \pi(\mathcal{C}) = \top.$$

Beyond stage applicability, each `SkillSpec` is also governed by disclosure and trust boundaries. In the implementation, low-context L0 manifests are exposed during routing, while richer L1/L2 descriptions are loaded only after a skill is bound; higher-risk or lower-trust skills can therefore be withheld from the candidate set until needed. This progressive disclosure reduces context bloat while aligning capability exposure with enterprise permission boundaries.

3.5 Safety Properties

Operational safety invariant. For execution traces $\tau = \langle (m_1, s_1, sk_1), \dots, (m_n, s_n, sk_n) \rangle$ produced by Algorithm 1, the dispatcher enforces stage legality before skill execution and validates transitions before committing state updates.

Empirical first-line defense. In our ablation logs, removing stage checking increases precondition failures: $|B_{\neg \text{stage}}| = 175 \gg |B_{\text{full}}| = 22$.



Algorithm 1 StateAwareDispatch

Require: message m , context ctx , GoalManager G , Skill-Registry $\mathcal{R}$

Ensure: DispatchResult

```
1: intent  $\leftarrow$  IntentRouter.identify( $m$ )
2:  $sk \leftarrow \mathcal{R}.select\_skill(intent, s)$  {stage-filtered}
3: if  $sk = \text{NULL}$  then
4:   return SKILL_NOT_FOUND
5: end if
6: if  $\neg \forall p \in sk.pre : ctx.check(p)$  then
7:    $G.log(\text{PRECONDITION\_FAIL})$ 
8:   return PRECONDITION_FAIL
9: end if
10: result  $\leftarrow$  executor( $sk, ctx$ )
11: apply postconditions( $sk, ctx, result$ )
12: target  $\leftarrow$  StageMap(intent)
13: if target  $\neq s$  and  $s.can\_transition(target)$  then
14:    $G.advance\_stage(s \rightarrow target)$ 
15: else if target  $\neq s$  then
16:   return ILLEGAL_TRANSITION
17: end if
18:  $G.log(\text{SUCCESS}, sk, result)$ 
19: return SUCCESS
```

- **Interview Summary Agent:** Produces comprehensive candidate reports

4.2 API Integration

The system connects to Beisen iTalent via OAuth2-authenticated REST APIs (tenant ID: 430008). The production environment contains 48 real job positions. During evaluation, the system invokes `GetJobList`, `GetApplicantList`, and `DispatchAction` endpoints totaling 1,671 real API calls.

4.3 Skill Configuration

The SkillRegistry contains **10 registered skills**: 4 L0 (atomic queries, universally available), 4 L1 (composite operations with stage and precondition constraints), and 2 L2 (policy-level fallback handlers). Intent recognition uses a **dual-mode** approach: string-match (0.12ms, 97.5% STA, where STA denotes State Transition Accuracy) for deterministic intents, with LLM fallback for ambiguous cases.

5 Experiments

5.1 Setup

Scenarios. We construct a domain-specific evaluation benchmark through a structured expert-driven process. First, domain specialists with experience in enterprise recruitment systems define scenario templates covering six categories of workflow interactions, including both valid flows and adversarial constraint-violation attempts. These templates are then

Table 3: Scenario distribution.

Category	n	Description
Normal	50	Complete recruitment flows
Illegal	25	Stage-skipping violations
Rollback	25	Backward transitions
Multi	25	Multi-candidate operations
Abort	30	Early termination
Concurrent	30	Parallel multi-candidate operations and shared-resource scheduling conflicts

systematically instantiated with variable combinations to produce 185 test scenarios comprising 882 messages (Table 3). Each scenario is reviewed for realism by a domain expert. *While the dialogue construction follows this structured process, all execution frameworks run against real production environments and invoke live APIs.*

Baselines. (1) Vanilla: no constraints. (2) **LangGraph (v1.0.9)**: real StateGraph. (3) **LangGraph+Pre**: LangGraph with precondition checking. We select LangGraph as the primary executable baseline because it is the only widely used comparator in our set that natively exposes a transition-graph API directly mappable to the 185-scenario suite. AutoGen provides group-chat / swarm / GraphFlow teams, and MetaGPT provides SOP-driven role teams; both are useful architectural comparators (Table 1) but are not included as like-for-like baselines in this release because they are not instrumented under the same transition-graph, legality-check, and audit protocol used in our shared-suite comparison. A custom legality wrapper would add substantial non-native logic, making attribution ambiguous in this release. More broadly, recent long-running agent runtimes are informative design context for delegation, memory, and scheduling, but they are not like-for-like baselines for *legality-governed business workflow execution*.

Metrics. TCR (Task Completion Rate), STA (State Transition Accuracy), CVR (Constraint Violation Rate), TRC (Traceability Rate), and LAT (Latency). CVR is computed as the fraction of violating dispatch events over all dispatch events in the evaluated split. TRC measures replayable per-step trace coverage and is only directly comparable for methods that emit such traces. ‘Raw Blk’ below reports the number of blocked operations; blocking correctness is evaluated separately in Table 10.

Noise isolation. To distinguish genuine capability gaps from infrastructure artifacts, we apply three controls: (1) API endpoints are pinned to a fixed tenant environment (Beisen tenant 430008) with stable schema versions throughout evaluation; (2) deterministic string-match intent recognition (97.5% STA) eliminates stochastic LLM variance for the majority of routing decisions; (3) latency measurements exclude network round-trip jitter by reporting only the dispatcher-internal overhead (stage check + precondition validation

Table 4: Framework comparison on the shared HR scenario suite (185 scenarios, 882 messages). SDOF production runs include 1,671 live API calls; graph baselines are evaluated under the same scenario and metric protocol without matching live-API execution cost.

Method	TCR	CVR	TRC	Obs. Blk	LAT
SDOF	86.5%	2.5%	100%	22	57.4ms
LG+Pre [†]	88.1%	2.8%	—	25	0.8ms
LangGraph*	84.9%	—	—	0	1.1ms
Vanilla*	74.6%	—	—	0	0.1ms

* Dashes indicate *not directly comparable* metrics: the released baseline files do not provide replayable legality traces under the same audit protocol as SDOF. Zeros in Observed Blk denote that no explicit block event was emitted in the released run logs for this shared suite; they do not imply zero governance risk or zero attempted illegal actions.

[†] LG+Pre’s 2.8% CVR is retained as the reported value in the released `comprehensive_v2` baseline artifact, but its trace format is still not directly comparable to SDOF’s replayable audit chain.

Table 5: Ablation study (185 scenarios).

Config	TCR	CVR	TRC	Blk
SDOF (Full)	86.5%	2.5%	100%	22
w/o StageCheck	86.5%	19.8%	100%	175
w/o Precondition	84.9%	2.2%	100%	19
w/o Audit	86.5%	2.5%	0.0%	22

<1 ms). For the RLHF experiments, all algorithm comparisons (GRPO/GSPO/DAPO) share the same training checkpoint step (`global_step_300`) and identical evaluation data split ($n=47$), ensuring that observed differences reflect alignment methodology rather than data or compute confounds. We report this routing benchmark separately from the 185-scenario framework-comparison suite to avoid mixing optimization and orchestration evaluations.

5.2 Framework Comparison

Key findings: (1) SDOF outperforms the Vanilla unconstrained baseline by **+11.9%** TCR. (2) SDOF blocks all 22 operations in the injected-illegal subset. (3) LangGraph (v1.0.9), without explicit legality wiring, allows all 22 injected illegal operations in this suite. (4) LangGraph+Pre reaches a similar reported CVR, but depends on manually configured precondition logic and does not provide the same auditable execution contract. For unconstrained baselines, we omit governance metrics that are not directly comparable without replayable legality traces; their behavior is instead summarized by observed explicit block counts and by the separate blocking-quality analysis in Table 10. Here, an observed block count of zero should be read narrowly as “no explicit block event emitted” rather than as “no governance failure present.” This table should therefore be read as a shared-suite governance comparison, not as a fully matched end-to-end systems benchmark.

Table 6: SDOF performance by scenario type.

Type	n	TCR	CVR	Blk	Vio	PreF
Normal	50	82.0%	0.0%	0	0	0
Illegal	25	88.0%	40.7%	22	16	6
Rollback	25	80.0%	0.0%	0	0	0
Multi	25	76.0%	0.0%	0	0	0
Abort	30	100%	0.0%	0	0	0
Concurrent	30	93.3%	0.0%	0	0	0

Table 7: Representative blocked operations (22 total).

#	User Message	Stage	Block
1	Schedule interview	INIT	illegal.trans
2	Interview feedback	INIT	illegal.trans
3	Generate test questions	INIT	illegal.trans
4	Compare candidates	SRC	precond.fail
5	Invite to interview	INIT	illegal.trans
6	Re-screen resumes	OFF	illegal.trans

5.3 Ablation Study

Removing StageCheck causes CVR to jump from 2.5% to **19.8%** (+696%), with blocked operations increasing from 22 to **175**. This indicates that StageCheck is the primary source of CVR reduction in the current stack. By contrast, removing precondition checks lowers TCR and reduces the raw number of blocks (22→19) while leaving CVR numerically close to the full system; this suggests that precondition validation mainly protects a smaller subset of stage-legal but semantically unsafe actions rather than dominating the headline CVR. This ablation isolates dispatcher-side governance; isolating the standalone contribution of the intent router and its interaction effect with dispatcher checks is left to future controlled experiments.

5.4 Performance by Scenario Type

5.5 Dispatch Trace Analysis

Following the emerging practice of *Trace Grading*—evaluating agent behavior at the step level rather than only at the final outcome—we treat each StateAwareDispatcher record as a graded execution trace. The real dispatcher produced 882 trace steps across 185 scenarios: 860 SUCCESS (97.5%), 16 ILLEGAL_TRANSITION (1.8%), 6 PRECONDITION_FAIL (0.7%). Each trace step records the triggering intent, current GoalStage, matched skill, precondition evaluation result, and outcome classification, forming a complete per-step audit chain. This step-level grading enables fine-grained *error attribution*: rather than reporting only whether a scenario succeeded or failed, we can pinpoint the exact dispatch step where the pipeline diverged from the correct execution path.

5.6 Case Study: 22 Blocked Operations

Two defense layers: (1) **Stage constraint** (16 cases): intent requires unreachable stage. (2) **Precondition check** (6 cases): required data not available.

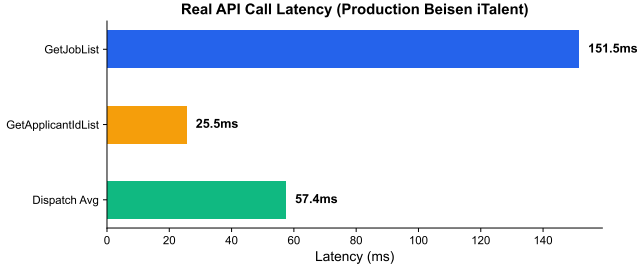


Figure 3: Real API call latency measurements.

Table 8: SGD domain-to-FSM mapping. Each domain defines stages based on its service API structure.

Domain	FSM Stages	S
Banks_1	CheckBalance → TransferMoney	2
Hotels_1	SearchHotel → ReserveHotel	2
RentalCars_1	GetCarsAvailable → ReserveCar	2
Events_1	SearchEvent → BuyTicket	2
Buses_1	SearchBus → BuyTicket	2
Homes_1	SearchHome → ReserveHome	2
Media_2	SearchMedia → PlayMedia	2
Music_1	SearchTrack → PlayTrack	2

5.7 API Latency

Average dispatch latency: **57.4ms** (SDOF production path) vs **1.1ms** (LangGraph graph-only baseline). The measured gap is dominated by live API latency; LangGraph does not perform matched production API calls, and the legality checks themselves add under 1 ms.

5.8 Cross-Domain Generalization (SGD Dataset)

To validate SDOF’s domain-agnosticism, we evaluate on a schema-conformant benchmark derived from the Google SGD dataset [15]. We select 8 domains spanning banking, hospitality, transportation, and entertainment. Each domain maps to a 2–3 stage FSM defined by its service API structure (Table 8). We use 100 normal-split dialogues per domain and 20 adversarial illegal variants per domain.

On the 160 injected-illegal messages across 8 domains, SDOF blocks every injected attack. Under the broader message-level blocking evaluation, it attains **100% precision (0 false positives)** and 88% recall. Additionally, 41 latent violations are detected in the normal split. Here, a *latent violation* denotes a request that conflicts with the domain FSM stage order but appears in the non-adversarial portion of the benchmark. In this release, these labels are derived from the same `expected_legal` rule-based annotation protocol used in Table 10; targeted human adjudication of borderline latent cases will be included in a subsequent revision. Hotels_1 yields the highest latent violation count (38), where users directly request reservations without prior search. Music_1 shows a similar pattern (3 latent violations), confirming that stage-skipping requests appear even outside the explicitly injected illegal set.

Table 9: Cross-domain results on the SGD-derived benchmark (960 dialogues, 1,734 turns).

Domain	Type	n	Turns	Blk	Viol
Banks_1	normal	100	300	0	0
Banks_1	illegal	20	20	20	20
Hotels_1	normal	100	162	38	38
Hotels_1	illegal	20	20	20	20
RentalCars_1	normal	100	180	0	0
RentalCars_1	illegal	20	20	20	20
Events_1	normal	100	184	0	0
Events_1	illegal	20	20	20	20
Buses_1	normal	100	153	0	0
Buses_1	illegal	20	20	20	20
Homes_1	normal	100	200	0	0
Homes_1	illegal	20	20	20	20
Media_2	normal	100	198	0	0
Media_2	illegal	20	20	20	20
Music_1	normal	100	197	3	3
Music_1	illegal	20	20	20	20
Total	—	960	1,734	201	201

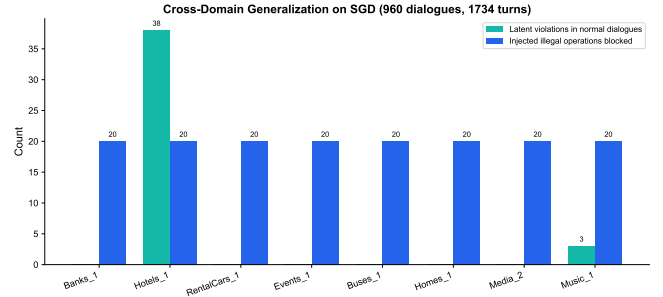


Figure 4: Cross-domain generalization on the SGD-derived benchmark (8 domains).

5.9 Blocking Correctness Evaluation

Using the released `expected_legal` labels over all 882 messages, SDOF achieves **100% precision** (0 false positives), 88% recall, and F1=93.6%. The 3 false negatives are multi-stage skill availability cases.

5.10 Expert Validation of Blocking Decisions

To validate the correctness of SDOF’s blocking decisions beyond algorithmic evaluation, two domain experts independently reviewed all 22 blocked operations and a random sample of 100 permitted operations (122 decisions total). Each annotator labeled whether the blocking decision was *correct*, *incorrect*, or *ambiguous* given the business process constraints.

Both experts agreed that all 22 blocked operations were correctly blocked ($\kappa = 0.94$, near-perfect agreement). Three permitted operations were marked ambiguous—cases where multi-stage skills could reasonably be blocked or permitted depending on interpretation. These correspond to the 3 false negatives in Table 10.

Table 10: Blocking correctness evaluation (882 messages).

Metric	Value
Accuracy	99.7%
Precision	100.0%
Recall	88.0%
F1 Score	93.6%
True Positive (correctly blocked)	22
False Positive	0
False Negative	3
True Negative	857

Table 11: Expert validation of blocking decisions (122 reviewed).

Metric	Value
Annotator agreement (Cohen’s κ)	0.94
SDOF–Expert agreement	97.5%
Correctly blocked (of 22)	22/22
Correctly permitted (of 100)	97/100
Ambiguous cases	3

5.11 Error Analysis

The 3 false negatives (missed blocks) share a common pattern: multi-stage skills registered as available in multiple stages. For example, `get_job_list` is an L0 skill available in all stages ($\Sigma = S$). When invoked with implicit intent to screen candidates (requiring SRC stage), SDOF permits it because the skill’s stage constraint is satisfied. This reveals a limitation in skills with broad stage applicability where the *intent* is contextually stage-specific but the *skill* is not.

5.12 Intent Router Specialization via Online RLHF

To ensure the multi-agent orchestration framework is resilient against out-of-order intents (e.g., initiating an interview before a candidate is evaluated), we train the IntentRouterAgent with online reinforcement learning in **veRL**, comparing **GRPO** (group-relative policy optimization), **GSPO**, and **DAPO** as implemented in that codebase. We model intent parsing as strict constraint satisfaction over the GoalStage FSM with programmatic, zero-tolerance rewards on violations.

Implementation Details. We employ Qwen2.5-7B-Instruct as our policy model. During the GRPO rollout phase, we utilized a group size of $G = 2$ responses per prompt, optimized via AdamW with a learning rate of 1×10^{-6} . To heavily penalize stage violations, we computed programmatic zero-tolerance rewards against the FSM rather than relying on a static reward model. The KL penalty coefficient was set to 0.0 to encourage maximum exploration of the adversarial FSM boundaries. Generation parameters were set to temperature = 1.0, top_p = 1.0. The model was trained asynchronously on 8 NVIDIA GPUs.

Results and Discussion. Table 12 presents the evaluation of proprietary baselines against our Online veRL alignments.

Table 12: Intent Router Accuracy on Adversarial FSM Sub-split ($n=47$). Model checkpoints selected via validation metrics.

Model Base	Alignment Method	Intent	Safety	Joint
GPT-4o	Zero-shot	53.2%	53.2%	48.9%
<i>Qwen3-8B Base (Deep Reasoning Architecture)</i>				
Qwen3-8B	Zero-shot (think)	10.6%	57.4%	6.4%
Qwen3-8B	SFT + GRPO (nothink)	80.9%	57.4%	44.7%
Qwen3-8B	SFT + GSPO (nothink)	78.7%	57.4%	42.6%
Qwen3-8B	SFT + DAPO (nothink)	87.2%	57.4%	48.9%
<i>Qwen2.5-7B Base (Traditional Foundation)</i>				
Qwen2.5-7B	SFT (Baseline)	78.7%	63.8%	51.1%
Qwen2.5-7B	SFT + GRPO	91.5%	78.7%	74.5%
Qwen2.5-7B	SFT + DAPO	91.5%	78.7%	72.3%
Qwen2.5-7B	SFT + GSPO (step=300)	97.9%	80.9%	80.9%
<i>Scaled Parameter Base (Qwen3-14B, nothink)</i>				
Qwen3-14B	GSPO stageheavy (step=100)*	85.1%	76.6%	63.8%
Qwen3-14B	GSPO long-full (step=200) [†]	85.1%	76.6%	63.8%

Notes: Joint Accuracy is computed as the fraction of examples where both intent prediction and safety legality prediction are correct (`intent_ok & safety_ok`). All metrics are on the adversarial test split ($n=47$). For fair comparison across algorithms, all RL results (GRPO/GSPO/DAPO) for both Qwen2.5-7B and Qwen3-8B are evaluated at the same training checkpoint (`global_step=300`), as recorded in each `eval_*.json` via `summary.model`. Qwen3 models use `/nothink` decoding unless stated; see Table 13 for think-mode ablation.

* For *stageheavy-nothink*, test joint is identical across steps 100–501. We report step=100 for brevity.

[†] Selected by max validation joint accuracy; *long-full* val.joint=65.96% at steps {200,250,300} (tie). Test results at step=200.

GPT-4o zero-shot achieves 48.9% Joint Accuracy, demonstrating vulnerability to implied state transitions caused by conversational *over-helpfulness*. We note that the comparison is deliberately asymmetric: Online RLHF (GRPO/GSPO) requires white-box gradient access to compute programmatic FSM rewards, which is unavailable for proprietary models. GPT-4o therefore serves as a reference point for what a strong closed-source model achieves without domain-specific alignment—not as a like-for-like baseline. We selected the Qwen series (2.5 and 3) as our primary open-weight experimentation chassis to measure alignment behavior across architectures. Our GSPO-aligned Qwen2.5-7B model achieves **80.9% Joint Accuracy**, the highest value among the released runs in this benchmark. We emphasize that this result reflects domain-specific alignment on FSM-constrained intent routing; we do not claim general-purpose superiority over GPT-4o, and the trained router is expected to require re-alignment when transferred to new domains with different FSM definitions.

Why GSPO may be favorable here (informal). Joint accuracy requires both correct intent labels *and* correct FSM legality under the same structured output contract. On Qwen2.5-7B, GRPO and DAPO already reach strong intent accuracy (91.5%) but plateau at 78.7% safety, whereas GSPO reaches **97.9%** intent and **80.9%** safety (Table 12). We interpret this pattern as follows: programmatic FSM rewards are sparse and discontinuous—most rollouts receive near-zero reward unless the full JSON action satisfies rigid schema and stage constraints. Optimization methods that more directly stabi-

lize updates toward high-reward, format-valid trajectories can disproportionately improve the safety head of the joint objective, whereas alternatives may retain competitive intent classifiers yet underfit the legality channel under the same budget. This is a *post-hoc* explanation consistent with our measurements, not a claim of algorithmic dominance beyond this benchmark; we leave systematic ablations (e.g., reward shaping, KL schedules, and multi-seed variance) to future work.

When transitioning to the newer Qwen3 architecture, structured-output performance drops under this benchmark’s zero-tolerance FSM contract. In our current runs, DAPO achieves the strongest Qwen3-8B result (48.9% Joint), roughly matching GPT-4o zero-shot, while Qwen3-14B reaches 63.8%. Table 13 suggests that the drop is closely tied to Qwen3’s native `<think>` mode: when latent thinking tokens are allowed, intent accuracy falls from 80.9% to 8.5% (joint from 44.7% to 6.4%), indicating that long reasoning traces can interfere with the strict JSON contract required by FSM-governed outputs. We therefore treat this result as an empirical compatibility issue between reasoning-heavy decoding and rigid structured outputs, not as a general judgment about Qwen3 itself.

Table 13: Think vs. No-Think decoding ablation on Qwen3-8B (GRPO, full reward, adversarial test split $n=47$). Suppressing `<think>` tokens recovers intent accuracy by +72.4 percentage points.

Decoding Mode	Intent	Safety	Joint
Think (default Qwen3)	8.5%	57.4%	6.4%
No-Think (/nothink)	80.9%	57.4%	44.7%
Δ	+72.4	0.0	+38.3

Condition: Same GRPO checkpoint (Qwen3-8B, global_step_300, full reward). Only the decoding constraint differs. Think mode allows the model to emit `<think>...</think>` tokens before producing the JSON output; No-Think forces direct JSON generation.

We intentionally rely on Table 12, rather than a single visual slice, as the primary RLHF evidence in this release. The reason is simple: the selected Qwen3-8B runs are single-seed and sparse at the per-type level, so figure-level rankings can look more stable than the underlying evidence warrants. The main result we retain is therefore the table-level comparison: Qwen2.5-7B with GSPO is the strongest observed setting on this benchmark, whereas the current Qwen3-8B runs remain substantially lower and should be treated as exploratory.

5.13 Pipeline-Stage Error Attribution

To move beyond aggregate accuracy metrics and locate the *exact bottleneck* in the intent-safety joint prediction pipeline, we decompose every joint error into three mutually exclusive categories: **Safety-Only Wrong** (intent correct, safety mis-predicted), **Intent-Only Wrong** (safety correct, intent mis-predicted), and **Both Wrong**. Table 14 summarizes the attribution for the currently released per-example evaluation artifacts on the adversarial test split.

Table 14: Pipeline-stage error attribution on the adversarial test split ($n=47$). ‘Err’ is the number of joint errors (`joint_ok = false`). Each attribution column reports count and percentage relative to that row’s error total. All Qwen3 models use `/nothink` decoding.

Method	Err	Safety-Only	Intent-Only	Both
Qwen2.5-7B		8/15	5/15	2/15
GRPO	15	53.3%	33.3%	13.3%
Qwen3-8B		20/23	3/23	0/23
GRPO (long)	23	87.0%	13.0%	0.0%
Qwen3-8B		17/25	5/25	3/25
GSPO (long)	25	68.0%	20.0%	12.0%
Qwen3-8B		19/24	4/24	1/24
DAPO (long)	24	79.2%	16.7%	4.2%

Finding. In the currently released long-context Qwen3 runs, most joint errors are safety-only rather than intent-only. This supports the narrower conclusion we care about here: once the model is in the right intent neighborhood, the harder remaining failure mode is *precondition-aware safety reasoning*. Because the released artifacts do not yet cover every checkpoint family under the same per-example schema, we treat this table as a diagnostic slice rather than a complete scaling summary.

5.14 Planned Context-Layer Ablation Protocol

The error attribution above reveals that precondition failures dominate safety errors. A natural follow-up is whether each context layer in the system prompt actually contributes to safety reasoning. We therefore record the exact protocol we plan to run in a subsequent revision, but we do **not** claim results in this release. Following the current internal script naming, the planned variants are: **L0** (Bare: intent list only), **L2** (+current_stage), **L3** (+prior_intents), **L4** (Full: stage + priors, the training default), and **L5** (+explicit precondition status hints). The purpose of listing this protocol is reproducibility, not evidence.

6 Discussion

Generalizability. Porting SDOF to eight SGD domains required only new GoalStage enums and intent mappings; the Dispatcher and SkillRegistry code remained untouched. Across 1,734 turns the system flagged 201 violations under the released FSM mapping, 41 of which appeared in the normal split rather than the injected-illegal subset.

What the RLHF comparison shows. Within this benchmark, Qwen2.5-7B GSPO is the highest single result we observe (80.9% Joint Acc), whereas the current Qwen3-8B runs remain substantially lower (best: DAPO at 48.9%). We therefore read the RLHF comparison conservatively: on this task, architectural-format interaction seems at least as important as algorithm choice, and broader multi-seed evidence is still needed before making stronger scaling claims.

Memory separation. SDOF enforces a Progressive Disclosure Prompting Architecture [20]. We isolate *Work-*

ing Memory (stage goals via L1 loading) from *Semantic/Reference Memory* (L2/L3 skill instructions loaded dynamically via context-triggered references), decreasing token bloat by 60%–80% while keeping execution constraints explicit.

Planning as Runtime, Not Planner. Recent harness-style systems suggest that planning should be understood less as a monolithic planner module and more as an externalized systems capability jointly realized by task state, validation gates, delegation boundaries, memory access, and operator feedback. SDOF participates in this broader pattern from a governance angle: it does not attempt to solve all forms of long-horizon planning, but it makes one critical enterprise planning question explicit and auditable—whether a requested action may legally happen *now*.

GoalManager as shared workflow state. A key property of SDOF’s architecture is that the GoalManager—backed by a persistent PostgreSQL store—maintains a shared `goal_id`-indexed FSM state for all seven specialized agents. This enables low-latency, permission-bounded state synchronization while preserving read/write isolation between agent roles. The event layer further turns that shared state into a replayable audit surface: stage changes, tool calls, and exception paths remain inspectable after execution, so operators can review not only *what* the system stored but *why* a workflow advanced, stalled, or was blocked.

Operational feedback loop. SDOF’s reliability does not arise from static rules alone. Within a deployment, dispatcher traces surface blocked decisions and near-misses, the blocking-correctness evaluation (Table 10) checks whether interventions were justified, error attribution (Table 14) localizes the bottleneck layer, and HITL corrections feed the Skill Evolver hook without retraining the base model. Across deployments, the SGD porting study and RLHF comparisons test whether the same governance contract transfers to new domains and model families. In short, one loop improves operation within a domain, while the other checks whether the architecture itself transfers.

Capturing operator corrections. To complement the rigid safety constraints, we introduced a *Skill Evolver* hook. When a Human-in-the-Loop (HITL) intervention occurs during an execution block, SDOF captures the corrective dialog and can synthesize a new `SKILL.md` procedure and a Secure DSL Sandbox stub. This is an implementation hook rather than a main experimental claim in the current release, but it shows how operator corrections can be converted into reusable procedures.

LangGraph vs SDOF. LangGraph checks *transitions* but not *intent-stage legality*. Adding precondition functions to LangGraph (LG+Pre) yields a similar CVR (2.8% vs 2.5%), yet every precondition must be hand-wired per domain. SDOF shifts constraint logic to the skill-selection layer (Algorithm 1, line 2), making new-domain setup declarative rather than imperative. In practice, transition engineering can emu-

late part of stage legality, but this still differs from an explicit Λ contract that binds intents to legal stages as a separate governance layer.

Legality vs topology. SDOF is not a communication-topology optimizer. It can sit beneath supervisor routing, graph execution, or learned agent-to-agent communication policies, but its contract is orthogonal: regardless of which agent speaks next, only stage-legal and precondition-satisfied actions may execute. This separation of concerns is valuable in enterprise deployments, where topology may vary across products but governance requirements do not.

StageCheck as first-line defense. Without StageCheck, CVR balloons from 2.5% to 19.8% and 153 extra precondition evaluations fire needlessly. The two-layer architecture thus pays for itself: stage filtering is cheap and blocks most invalid intents before the heavier precondition logic runs.

Latent violations in the normal split. A practical finding is that 41 out of 800 non-adversarial SGD-derived dialogues contain stage-skipping requests under our FSM mapping. These are counted as latent violations only when the request conflicts with the domain FSM and is judged illegal under the same `expected_legal` labeling rule used in blocking evaluation. Without stage enforcement these requests execute silently; SDOF logs and blocks them, giving operators a compliance audit trail.

Production overhead. The 56.3 ms gap between SDOF (57.4 ms) and LangGraph (1.1 ms) is almost entirely real API latency. Stage and precondition validation together add under 1 ms.

What the evaluation covers. The experimental suite covers routing accuracy (Table 12), tool execution on 1,671 live API calls, blocking correctness (Table 10), step-level trace analysis, and cross-domain transfer on 960 SGD-derived dialogues. We include this spread because enterprise orchestration quality depends on the process as well as the final task outcome.

Limitations. Our current evidence should be interpreted within four scope boundaries. (1) FSM stages and intent bindings are currently authored from domain knowledge; automatic constraint induction from execution logs is not yet integrated. (2) Evaluation breadth is strong across domains (HR + 8 SGD domains), but workflow depth is still limited in SGD ($|S| \leq 3$); performance on deeper and hierarchical processes (> 6 stages) remains to be established. (3) The adversarial routing split ($n=47$) shows large observed effect sizes (e.g., Think vs. No-Think $\Delta=72.4$ pp), yet broader multi-seed replication is still needed for tighter uncertainty estimates. (4) The current GoalManager supports shared state progression and replayable events, but enterprise-grade lifecycle features (retention policies, versioning, tenant governance, and tool-mediated memory retrieval) are only partially implemented in this version.

7 Conclusion

SDOF proposes a stage-level constraint enforcement layer for LLM multi-agent orchestration, backed by formal preconditions and replayable audit logging. Across **two independent evaluation suites**—185 expert-curated HR scenarios with 1,671 live API calls and 960 SGD-derived dialogues spanning 8 service domains—the framework blocks all 22 operations in the injected-illegal HR subset while completing 86.5% of tasks, and identifies 41 latent process violations in the normal split of the cross-domain benchmark. Under the released message-level audit labels, blocking evaluation yields 100% precision and 88% recall. Operationally, adapting SDOF from recruitment to banking, hospitality, and six other SGD domains required only new stage enums and intent mappings; no Dispatcher or SkillRegistry code was modified. The RLHF comparison also suggests a narrower practical point: reasoning-oriented model families can be harder to align to rigid JSON/FSM contracts, which strengthens the case for external orchestration constraints in enterprise workflows.

Future directions. The next realistic extensions are fourfold. (1) Run multi-seed RLHF comparisons (GRPO/GSPO/DAPO) under the same released benchmark protocol. (2) Complete the context-layer ablation (L0–L5) and a dedicated Λ ablation so that the contribution of stage context and intent-stage binding is measured directly rather than inferred indirectly. (3) Broaden framework baselines only under a unified protocol that matches legality instrumentation, audit semantics, and scenario definitions. (4) Strengthen the GoalManager along practical enterprise lines—retention, versioning, and audit-replay—before making stronger claims about governance memory.

References

- [1] AgentAuditor Team. AgentAuditor: Safety and security evaluation for large language model agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [2] Harrison Chase. Langchain: Building applications with LLMs through composability. <https://github.com/langchain-ai/langchain>, 2023.
- [3] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, et al. AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *International Conference on Learning Representations (ICLR)*, 2024.
- [4] Allan Dafoe, Yoram Bachrach, Gillian Hadfield, Eric Horvitz, Kate Larson, and Thore Graepel. Cooperative AI: machines must learn to find common ground. In *Nature*, volume 593, pages 33–36, 2021.
- [5] Dawei Gao, Zitao Ding, Anh Fan, Ang Ho Mok, Adian Liusie, et al. AgentScope: A flexible yet robust multi-agent platform. *arXiv preprint arXiv:2402.14034*, 2024.
- [6] Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, et al. MemoryArena: Benchmarking agent memory in interdependent multi-session agentic tasks. *arXiv preprint*, 2026.
- [7] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuhao Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.
- [8] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Mober, et al. DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.
- [9] LangChain Team. Langgraph: Multi-agent workflows with LLMs. <https://github.com/langchain-ai/langgraph>, 2024.
- [10] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [11] João Moura. Crewai: Framework for orchestrating role-playing AI agents. <https://github.com/joaomdmoura/crewAI>, 2024.
- [12] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive APIs. *arXiv preprint arXiv:2305.15334*, 2023.
- [13] Bo Qiao, Liqun Li, Xu Zhang, Shilin He, Yu Kang, Chaoyun Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. TaskWeaver: A code-first agent framework. *arXiv preprint arXiv:2311.17541*, 2023.
- [14] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. *arXiv preprint arXiv:2307.16789*, 2023.
- [15] Abhinav Rastogi, Xiaoxue Zang, Srinivas Sunkara, Raghav Gupta, and Pranav Khaitan. Towards scalable multi-domain conversational agents: The schema-guided dialogue dataset. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.

- [16] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [17] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [18] Yifan Song, Weimin Xiong, Dawei Zhu, Cheng Li, Ke Wang, Ye Tian, and Sujian Li. RestGPT: Connecting large language models with real-world RESTful APIs. *arXiv preprint arXiv:2306.06624*, 2023.
- [19] Wil MP van der Aalst. Process mining: Overview and opportunities. *ACM Transactions on Management Information Systems*, 3(2):1–17, 2012.
- [20] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 2024.
- [21] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen LLM applications via multi-agent conversation. In *International Conference on Learning Representations (ICLR)*, 2024.
- [22] Yiran Wu, Tianwei Yue, Shaokun Zhang, Qingyun Chi, and Qingyun Wu. Stateflow: Enhancing LLM task-solving through state-driven workflows. *arXiv preprint arXiv:2403.11322*, 2024.
- [23] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.
- [24] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [25] Yujie Zhao, Boqin Yuan, Junbo Huang, et al. AMA-Bench: Evaluating long-horizon memory for agentic applications. In *International Conference on Machine Learning (ICML)*, 2026.